



BENEFIT In-App iOS Documentation

Version 1.0.9 • 23 June 2025

FOO OFFSHORE SAL – www.foo.mobi

Copyright © FOO OFFSHORE SAL, 2017. All rights reserved.

The copyright in this document is owned by FOO OFFSHORE SAL, ["FOO"]. This document may not be reproduced, in whole or in part, in any form without the express consent of FOO in writing.

Information contained in this document is proprietary and confidential to FOO. That information, irrespective of form, must not be used other than for the purposes for which it is disclosed to the recipient and must not under any circumstances be disclosed to any third party without the express consent in writing of FOO.

Although FOO uses all reasonable efforts to ensure the accuracy and completeness of this document, no warranty or representation whatever is given by FOO in respect of it and any use of or reliance on any of the information contained herein is entirely at the risk the person so acting. FOO shall have no liability whatsoever in respect of any use of or reliance on any of such information.

FOO

Table of Contents

1. Overview	5
2. Framework Integration.....	6
3. Implementation	7
4. Deep Linking Configuration.....	8

Document History

Revision History

This document has been revised by:

Revision Number	Revision Date	Summary of Changes
V 1.0	15-01-2018	Initial version
V 1.0.1	26-07-2018	Allowing Merchants to send the transaction's Reference id.
V 1.0.3	12-09-2019	- Hashing Fix
V 1.0.4	13-09-2019	- New Hashing Method
V 1.0.5	21-11-2019	- Amount Checking
V 1.0.6	23-04-2024	- XCframework that works on iOS and Simulator - Bitcode Not Needed Anymore, disabled by default - No Need for Script Phase in New Phases in Settings
V 1.0.7	18-10-2024	- Minimum iOS Version 15.6 - iOS 18 Support - Moved to the new openUrl function in favour of the deprecated one.
V 1.0.8	23-10-2024	- Decimal amount issue fix.
V 1.0.9	23-6-2025	- Reverting Decimal Amount Checks to pass it as received

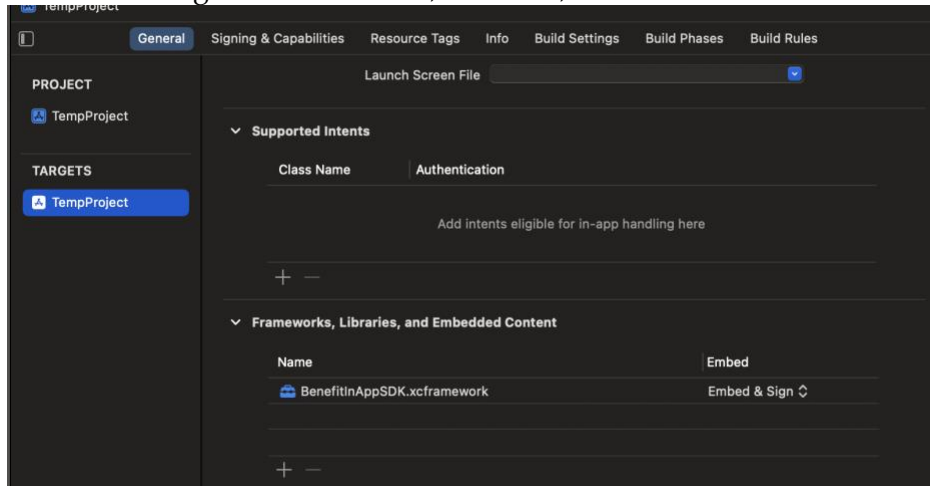
1. Overview

This document shows the technical specifications of Benefit In App solution. The SDK will be delivered in .XCFramework file format.

The purpose of Benefit In App SDK is to handle the communication between any merchant app and BenefitPay App. The user will be able to pay by simply clicking on the BenefitPay button in the corresponding merchant app where he will be redirected to the BenefitPay app and proceed with the payment flow (e.g. choosing a card, entering the pin code ...).

2. Framework Integration

- Add BenefitInAppSDK.xcframework to your project
- Go to “General” Settings of your project and change BenefitInAppSDK.xcframework to Embed & Sign in Frameworks, Libraries, and Embedded Content



- In the old documentation you were asked to add a script in Build Phases -> Remove Unused Architectures Script. This should be removed; the framework should work on Simulator and Real Device out of the box.

3. Implementation

- Add a UIView of size 258W 60H in your view controller and subclass it to BPinAppButton.
- Import <BenefitInAppSDK/BenefitInAppSDK.h> into your view controller.
- Link the delegate of this view to your view controller, or create an IBOutlet of it and add self.bpButton.delegate = self to your view controller.
- In your view controller add the following function:

```
- (BPinAppConfiguration *)bpInAppConfiguration
{
    BPinAppConfiguration *configuration = [[BPinAppConfiguration alloc] initWithAppld:@"12345"
andSecretKey:@"123456 andAmount:@"10" andCurrencyCode:@"048"
andMerchantId:@"12345678" andMerchantName:@"DominosPizza" andMerchantCity:@"Sadad"
andCountryCode:@"BH" andMerchantCategoryId:@"1234 andReferenceId:@"445544"
andCallbackTag:@"tempBN"];

    return configuration;
}
```

Fields	Type	Description
Appld	String	Provided by FOO
merchantId	String	Provided by FOO
secretKey	String	Provided by FOO
amount	String	Amount
currencyCode	String	Currency Code (e.g. for “BHD”, “048” should be sent)
merchantCategoryCode	String	As defined by ISO 8583:1993 for Card Acceptor Business Code
merchantName	String	Merchant Name
merchantCity	String	City of operations for the merchant
callbackTag	String	String to make Benefitpay App know where should go after a transaction
countryCode	String	ISO “Alpha-2” Code
referenceId	String	Generated Reference Id

NOTE: These properties can’t be nil or empty string

4. Deep Linking Configuration

- In order to get the results of the transaction, you should do the following in your AppDelegate.m:
 - Import <BenefitInAppSDK/BenefitInAppSDK.h>
 - Add BPDLPaymentCallBackItem *item = [[BPDLPaymentCallBackItem alloc] initWithDeepLinkURL:url]; to application:openURL Function:

```
- (BOOL)application:(UIApplication *)application openURL:(NSURL*)url sourceApplication:(NSString*)sourceApplication annotation:(id)annotation
{
    BPDLPaymentCallBackItem *item = [[BPDLPaymentCallBackItem alloc] initWithDeepLinkURL:url];
    return YES;
}
```

BPDLPaymentCallBackItem:

Fields	Type	Description
Status	Enum	Success, Cancel or Fail
MerchantName	String	Merchant Name
CardNumber	String	Last 4 digits of the selected card
Currency	String	Currency Abbreviation
CurrencyCode	String	Currency Code
Amount	String	Amount
Message	String	Transaction Fail Message. Nil if cancel or success
ReferenceId	String	Reference Id

- Go to your project “Info” and add the same CallbackTag used in section [3] under URL Types. URL Schemes. Keep identifier and icon empty, and change the role to none. Or you can add it in your “Info.plist”
- To make your app open BenefitPay when the button is clicked you should add the following in your “Info.plist”:

```
<key>LSApplicationQueriesSchemes</key>
<array>
    <string>benefitinapp</string>
</array>
```

For Swift Implementation with App Delegate you can use:

```
public func application(_ app: UIApplication, open url: URL, options:
[UIApplication.OpenURLOptionsKey : Any] = [:]) -> Bool {
    if let transaction = BPDFLPaymentCallbackItem.init(deepLinkURL: url) {
        debugPrint(transaction.referenceId)
    }

    return true
}
```

For Swift Implementation with Scene Delegate you can use:

```
func scene(_ scene: UIScene, openURLContexts URLContexts: Set<UIOpenURLContext>)
{
    if let url = URLContexts.first?.url,
        let transaction = BPDFLPaymentCallbackItem(deepLinkURL: url) {
        debugPrint(transaction.referenceId)
    }
}
```

For Swift Implementation with Scene Delegate you can use:

5. PaymentCallbackStatus

PaymentCallbackStatus refers to the callback returned to the InAppSDK from the returning deeplink “status” field:

1. If the value returned is “cancel” then we will have the status as:
PaymentCallbackStatusCancel
2. If the value returned is “fail” then we will have the status as: PaymentCallbackStatusFail
3. If the value returned is “success” then we will have the status as:
PaymentCallbackStatusSuccess
4. By Default, if the status is not returned or its value is missing the value will be
PaymentCallbackStatusFail